

Relatório sobre a atividade “20 - Prática: Reconhecimento de Emoções com TensorFlow 2.0 e Python (III)”

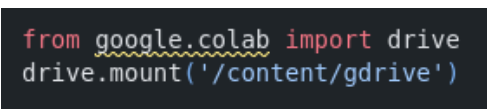
Lucas Gabriel Arenhardt

1. Introdução

Neste módulo foi realizada a aplicação prática dos conhecimentos adquiridos em aulas anteriores, utilizando deles para implementar uma rede neural capaz de realizar o reconhecimento de emoções. Para tal, foi utilizada a linguagem Python, juntamente do TensorFlow e OpenCV.

2. Desenvolvimento

Alguns conceitos iniciais são utilizados em todos os notebooks, como a importação de bibliotecas, a escolha da versão do TensorFlow através do código “%tensorflow_version 2.x” (embora não seja mais necessário) e a conexão do Google Drive com o Google Colab.



```
from google.colab import drive
drive.mount('/content/gdrive')
```

Imagem 1 - Código para conectar ao Google Drive.

Primeiramente, é realizado um teste utilizando modelos prontos de detecção de faces e reconhecimento de emoções. O código dessa parte consiste em carregar os modelos e aplicá-los em uma imagem. A imagem é manipulada para garantir que apenas as informações necessárias sejam passadas aos modelos. Para isso, a imagem é convertida para a escala de cinza utilizando a função `cv2.cvtColor(original, cv2.COLOR\_BGR2GRAY)`. Em seguida, a foto é cortada para manter apenas a face, que é a região de interesse (ROI). Posteriormente, os valores dos pixels são transformados para o tipo float e normalizados. Para finalizar a manipulação da imagem, é adicionada uma dimensão extra ao "roi", com a finalidade de deixá-lo no formato adequado para o TensorFlow. Após essa manipulação, os modelos são aplicados para gerar uma previsão para a imagem. Por fim, a bounding box é desenhada ao redor do rosto identificado, juntamente com um texto especificando a previsão da emoção feita. A imagem de um gráfico que indica a probabilidade de cada emoção também é gerada.

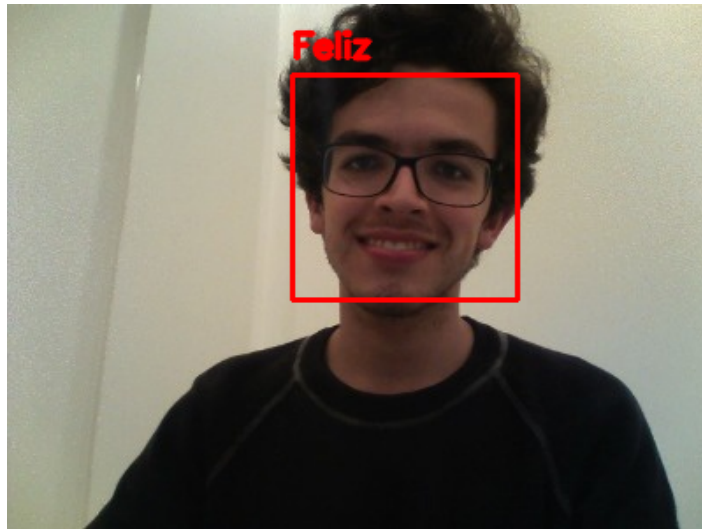


Imagem 2 - Resultado da aplicação dos modelos.

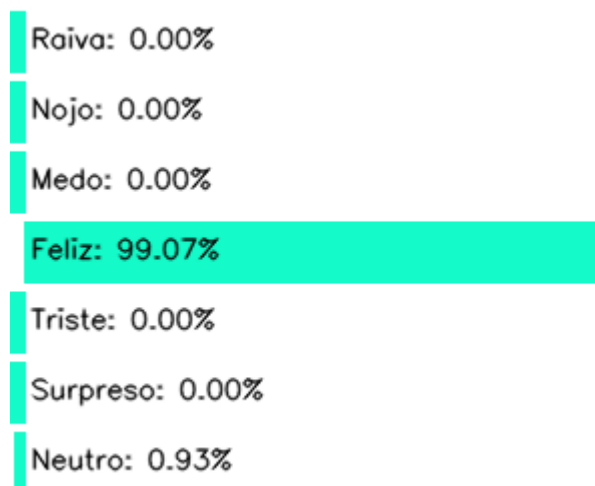


Imagem 3 - Gráfico que indica a probabilidade de cada emoção.

Na seção seguinte, a detecção de emoções é realizada através da construção de uma rede neural convolucional. Para isso, é carregado e convertido para um dataframe um arquivo csv que contém os valores dos pixels de várias imagens, assim não sendo necessário carregá-las no formato jpg ou png. A coluna referente aos pixels das imagens é transformada em uma lista e posteriormente dividida para formar as figuras separadamente. Além disso, também é aplicado um redimensionamento para deixá-las menores, poupando poder computacional. Como há 7 saídas possíveis para a rede neural, os valores referentes às emoções são transformados para o formato one-hot encoded.

Feito o pré-processamento, deve-se realizar a divisão dos dados em conjuntos de treinamento e teste utilizando a função “train_test_split()”. Nesse caso, foi criado também um conjunto de validação que será utilizado para avaliar cada época do treinamento.

A arquitetura da rede neural consiste em camadas de convolução, normalização, pooling, dropout, flattening e densas.

```
#camada de convolução
model.add(Conv2D(num_features, kernel_size=(3,3), activation='relu',
input_shape=(width, height, 1), data_format = 'channels_last',
kernel_regularizer = l2(0.01)))
#camada de convolução
model.add(Conv2D(num_features, kernel_size=(3,3), activation='relu', padding='same'))
#camada de normalização
model.add(BatchNormalization())
#camada de pooling
model.add(MaxPooling2D(pool_size=(2,2), strides=(2,2)))
#camada de dropout
model.add(Dropout(0.5))

model.add(Conv2D(2*num_features, kernel_size=(3,3), activation='relu', padding='same'))
model.add(BatchNormalization())
model.add(Conv2D(2*num_features, kernel_size=(3,3), activation='relu', padding='same'))
model.add(BatchNormalization())
model.add(MaxPooling2D(pool_size=(2,2), strides=(2,2)))
model.add(Dropout(0.5))

model.add(Conv2D(2*2*num_features, kernel_size=(3,3), activation='relu', padding='same'))
model.add(BatchNormalization())
model.add(Conv2D(2*2*num_features, kernel_size=(3,3), activation='relu', padding='same'))
model.add(BatchNormalization())
model.add(MaxPooling2D(pool_size=(2,2), strides=(2,2)))
model.add(Dropout(0.5))

model.add(Conv2D(2*2*2*num_features, kernel_size=(3,3), activation='relu', padding='same'))
model.add(BatchNormalization())
model.add(Conv2D(2*2*2*num_features, kernel_size=(3,3), activation='relu', padding='same'))
model.add(BatchNormalization())
model.add(MaxPooling2D(pool_size=(2,2), strides=(2,2)))
model.add(Dropout(0.5))

#aplicando o flattening
model.add(Flatten())

#camadas densas
model.add(Dense(2*2*2*num_features, activation='relu'))
model.add(Dropout(0.4))
model.add(Dense(2*2*num_features, activation='relu'))
model.add(Dropout(0.4))
model.add(Dense(2*num_features, activation='relu'))
model.add(Dropout(0.5))

model.add(Dense(num_labels, activation = 'softmax'))

model.summary()
```

Imagem 4 - Estrutura da Rede Neural Convolucional.

Vale destacar que algumas outras arquiteturas também são mostradas durante as aulas, porém o fundamento de todas é o mesmo.

Em seguida, o modelo é compilado utilizando a função de perda “categorical_crossentropy” e o otimizador “Adam”. Algumas funções de callback são implementadas, sendo: “ReduceLROnPlateau”, responsável por alterar a taxa de aprendizagem quando necessário;

“early_stopper”, interrompe o treinamento antes da terminarem as épocas caso o modelo não esteja progredindo; e “ModelCheckpoint”, que salva o modelo com as melhores métricas durante o treinamento. Por fim, é realizado o treinamento da rede neural.

Os resultados da implementação são dispostos em uma matriz de confusão gerada utilizando o matplotlib, como já realizado anteriormente no Bootcamp.

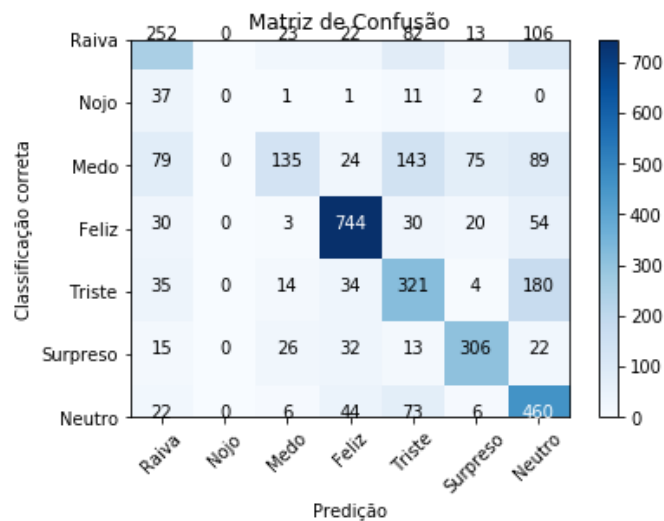


Imagem 5 - Matriz de Confusão.

O curso também aborda a aplicação do detector de emoções em vídeos. O processo é muito semelhante ao anterior, já que um vídeo nada mais é que um conjunto de imagens.

3. Conclusão

Pode-se destacar que, ao longo deste módulo, foi possível consolidar os conhecimentos adquiridos em etapas anteriores do Bootcamp, aplicando-os na implementação de um modelo de reconhecimento de emoções utilizando redes neurais convolucionais com TensorFlow. Além disso, as funções de Callback aprendidas abrem espaço para novas possibilidades durante o desenvolvimento da rede neural, possibilitando realizar ainda mais aprimoramentos.

Por fim, esta atividade foi crucial para compreender como técnicas de visão computacional e redes neurais podem ser eficazmente aplicadas para resolver problemas reais, como o reconhecimento de emoções.